

第二次作业

张航玮 2023141460185

```
1. import torch
2. import torch.nn as nn
3. import torch.optim as optim
4. import matplotlib.pyplot as plt
5.
6. import matplotlib.pyplot as plt
7. from matplotlib import rcParams
8.
9. rcParams['font.sans-serif'] = ['SimHei'] # 用黑体显示中文
10. rcParams['axes.unicode_minus'] = False # 正常显示负号
11.
12.
13.
14. # 假设我们有两个特征，用于二分类
15. torch.manual_seed(0)
16. num_samples = 200
17.
18. # 生成两个类别的数据
19. x_class0 = torch.randn(num_samples // 2, 2) + torch.tensor([2.0, 2.0]) # 类0分布在(2,2)附近
20. x_class1 = torch.randn(num_samples // 2, 2) + torch.tensor([-2.0, -2.0]) # 类1分布在(-2,-2)附近
21.
22. X = torch.cat([x_class0, x_class1], dim=0)
23. y = torch.cat([torch.zeros(num_samples // 2), torch.ones(num_samples // 2)], dim=0).view(-1, 1)
24.
25.
26. indices = torch.randperm(num_samples)
27. X, y = X[indices], y[indices]
28.
29. #定义逻辑回归模型
30. class LogisticRegression(nn.Module):
31.     def __init__(self, input_dim):
32.         super(LogisticRegression, self).__init__()
33.         self.linear = nn.Linear(input_dim, 1)
34.
35.     def forward(self, x):
36.         return torch.sigmoid(self.linear(x))
37.
38. model = LogisticRegression(input_dim=2)
39.
40.
41. criterion = nn.BCELoss() # 二分类交叉熵
42. optimizer = optim.SGD(model.parameters(), lr=0.1)
```

```
43.
44. # 训练
45. num_epochs = 100
46. for epoch in range(num_epochs):
47.
48.     outputs = model(X)
49.     loss = criterion(outputs, y)
50.
51.
52.     optimizer.zero_grad()
53.     loss.backward()
54.     optimizer.step()
55.
56.     if (epoch+1) % 10 == 0:
57.         print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')
58.
59. # 可视化结果
60. # 预测标签
61. predicted = (model(X) >= 0.5).float()
62.
63. # 计算准确率
64. accuracy = (predicted == y).sum().item() / y.size(0)
65. print(f'最终准确率: {accuracy * 100:.2f}%',)
66.
67. # 画出分类边界
68. x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
69. y_min, y_max = X[:,1].min()-1, X[:,1].max()+1
70. xx, yy = torch.meshgrid(torch.linspace(x_min, x_max, 100), torch.linspace(y_min, y_max, 100))
71. grid = torch.cat([xx.reshape(-1,1), yy.reshape(-1,1)], dim=1)
72. with torch.no_grad():
73.     zz = model(grid).reshape(xx.shape)
74.
75. plt.figure(figsize=(6,6))
76. plt.contourf(xx, yy, zz, levels=[0,0.5,1], alpha=0.2, colors=['blue', 'red'])
77. plt.scatter(X[:,0], X[:,1], c=y.squeeze(), cmap='bwr', edgecolors='k')
78. plt.title("逻辑回归分类结果")
79. plt.xlabel("x1")
80. plt.ylabel("x2")
81. plt.show()
```

